

A Cascaded ReRAM-based Crossbar Architecture for Transformer Neural Network Acceleration

JIAHONG XU, Huazhong University of Science and Technology, Wuhan, China

HAIKUN LIU, Huazhong University of Science and Technology, Wuhan, China

XIAOYANG PENG, Huazhong University of Science and Technology, Wuhan, China

ZHUOHUI DUAN, Huazhong University of Science and Technology, Wuhan, China

XIAOFEI LIAO, Huazhong University of Science and Technology, Wuhan, China

HAI JIN, Huazhong University of Science and Technology, Wuhan, China

Emerging *resistive random-access memory* (ReRAM) based processing-in-memory (PIM) accelerators have been increasingly explored in recent years because they can efficiently perform *in-situ matrix-vector multiplication* (MVM) operations involved in a wide spectrum of artificial neural networks. However, there remain significant challenges to apply existing ReRAM-based PIM accelerators to the most popular Transformer neural networks. Since Transformers involve a series of *matrix-matrix multiplication* (MatMul) operations with data dependencies, they should write intermediate results of MatMuls to ReRAM crossbar arrays for further processing. Conventional ReRAM-based PIM accelerators often suffer from high latency of ReRAM writes and intra-layer pipeline stalls.

In this paper, we propose ReCAT, a ReRAM-based PIM accelerator designed particularly for Transformers. ReCAT exploits *transimpedance amplifiers* (TIAs) to cascade a pair of crossbar arrays for MatMul operations involved in the self-attention mechanism. The intermediate result of a MatMul generated by one crossbar array can be directly mapped to another crossbar array, avoiding costly analog-to-digital conversions. In this way, ReCAT allows MVM operations to overlap with the corresponding data mapping, hiding the high latency of ReRAM writes. Furthermore, we propose an *analog-to-digital converter* (ADC) virtualization scheme to dynamically share scarce ADCs among a group of crossbar arrays, and thus significantly improve the utilization of ADCs to eliminate the performance bottleneck of MVM operations. Experimental results show that ReCAT achieves 207.3 $\times$, 2.11 $\times$, and 3.06 $\times$ performance improvement on average compared with other Transformer acceleration solutions—GPUs, ReBert, and ReTransformer, respectively.

CCS Concepts: • **Computer systems organization** → **Analog computers**; • **Hardware** → **Analog and mixed-signal circuit optimization**;

This work is supported jointly by National Key Research and Development Program of China under grant No. 2022YFB4500303, and National Natural Science Foundation of China (NSFC) under grants No. 62072198, 62332011, Natural Science Foundation of Hubei Province under grant No. 2021CFA037. All authors are with National Engineering Research Center for Big Data Technology and System, Service Computing Technology and System Lab, Cluster and Grid Computing Lab, School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan, China.

Authors' Contact Information: Jiahong Xu, Huazhong University of Science and Technology, Wuhan, Hubei, China; e-mail: jhxu@hust.edu.cn; Haikun Liu (Corresponding author), Huazhong University of Science and Technology, Wuhan, Hubei, China; e-mail: hkliu@hust.edu.cn; Xiaoyang Peng, Huazhong University of Science and Technology, Wuhan, Hubei, China; e-mail: xiaoyangpeng@hust.edu.cn; Zhuohui Duan, Huazhong University of Science and Technology, Wuhan, Hubei, China; e-mail: zhduan@hust.edu.cn; Xiaofei Liao, Huazhong University of Science and Technology, Wuhan, Hubei, China; e-mail: xfliao@hust.edu.cn; Hai Jin, Huazhong University of Science and Technology, Wuhan, Hubei, China; e-mail: hjin@hust.edu.cn.



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike International 4.0 License.

© 2024 Copyright held by the owner/author(s).

ACM 1084-4309/2024/12-ART10

<https://doi.org/10.1145/3701034>

Additional Key Words and Phrases: Transformer, ReRAM, PIM, analog-to-digital conversion

ACM Reference Format:

Jiahong Xu, Haikun Liu, Xiaoyang Peng, Zhuohui Duan, Xiaofei Liao, and Hai Jin. 2024. A Cascaded ReRAM-based Crossbar Architecture for Transformer Neural Network Acceleration. *ACM Trans. Des. Autom. Electron. Syst.* 30, 1, Article 10 (December 2024), 23 pages. <https://doi.org/10.1145/3701034>

1 Introduction

Transformer neural networks have achieved great success in handling various sequence-to-sequence tasks, such as **natural language processing (NLP)** [10, 24, 30] and computer vision [11, 14, 41]. However, the ever-growing scale of Transformer models leads to severe “memory-wall” and “power-wall” problems in traditional Von Neumann architectures due to the tremendous data movement between CPU and main memory. To mitigate these issues, several **processing-in-memory (PIM)** architectures [23, 28, 42, 48, 50] using **resistive random access memory (ReRAM)** are proposed since ReRAM-based crossbar architectures have demonstrated significant performance and energy efficiency in the field of artificial neural networks [6, 7, 31, 35, 37].

In a ReRAM-based PIM architecture, ReRAM cells in crossbar arrays are used to store elements of weight matrices as conductance values in advance. Then, a vector of input features is encoded as voltage values and applied to ReRAM crossbar arrays. In this way, these crossbar arrays can perform *in-situ* **matrix-vector multiplications (MVMs)** in the analog domain based on Kirchhoff’s circuit laws. Since an MVM operation can be finished within a constant time ($O(1)$ time complexity), ReRAM-based PIM architectures can significantly improve the performance and energy efficiency of many neural networks such as **convolutional neural networks (CNNs)** and **recurrent neural networks (RNNs)** [20]. However, there still remain several challenges when we directly adopt these PIM architectures for Transformers.

First, Transformers usually cause costly ReRAM write operations since the intermediate results are dynamically generated and should be written to crossbar arrays for further processing during self-attention operations. Unlike other **deep neural networks (DNNs)** in which all weight matrices are fixed and can be mapped to crossbar arrays in advance, storing the intermediate results is on the critical path of dataflow in Transformers. Since a matrix can only be mapped into a crossbar array column by column or row by row, mapping intermediate matrices into crossbar arrays would cause rather high latency and usually stall the computation [50]. Recently, a number of studies have been proposed to address this problem. For example, ReTransformer [50] tries to reduce the number of write operations and overlap the write latency with other ongoing analog MVMs. ATT [16] uses CMOS-based processing units to avoid ReRAM write operations for intermediate MVMs. Yang et al. [48] use analog signal memory and analog multiply-add circuits to eliminate ReRAM write operations for intermediate results. However, these proposals either fail to hide the ReRAM write latency completely or lower the computation parallelism due to using CMOS circuits.

Second, **Analog-to-Digital (AD)** conversions for intermediate analog results also cause significant performance degradation. In previous ReRAM-based PIM architectures, a crossbar array is typically equipped with many **digital-to-analog converters (DACs)** and **analog-to-digital converters (ADCs)** for signal conversions. The intermediate analog outputs generated from one crossbar array should be first converted into digital values, and then converted into analog values (conductance) again when they are written into another crossbar array. Actually, such back-and-forth signal conversions are unnecessary when **transimpedance amplifiers (TIAs)** [8] are adopted to convert accumulated currents into voltage values directly. However, it is challenging

to handle signed operands when the outputs of TIAs are directly written into crossbar arrays (Section 2.3).

Third, the scarce ADC resource would be underutilized when intermediate results are written into crossbar arrays. Since ADCs and crossbar arrays are tightly coupled, the ADCs associated with a crossbar array cannot be used when it is being programmed, resulting in a low utilization of the ADC resource. A few recent studies [23, 50] have been proposed to mitigate pipeline stalls due to ReRAM write operations, but overlook the low utilization of the ADC resource. As the AD conversion is a performance bottleneck in ReRAM-based PIM architectures [13, 27, 35, 43], the ADC utilization has a significant impact on the performance and energy efficiency of ReRAM-based crossbar arrays [46]. Thus, an ADC resource-sharing scheme is essential to improve the ADC utilization when intermediate results are being written into crossbar arrays for Transformers.

In this paper, we propose ReCAT, a ReRAM-based PIM architecture for Transformer neural networks. ReCAT is composed of conventional crossbar arrays and cascaded crossbar arrays. The former is associated with DACs and ADCs, while the latter uses *transimpedance amplifiers* (TIAs) to cascade pairs of crossbar arrays. We advocate three novel designs to address the aforementioned challenges.

- We propose cascaded crossbar arrays to hide the ReRAM write latency for intermediate results generated by the self-attention operations in Transformer neural networks. ReCAT exploits *transimpedance amplifiers* (TIAs) to cascade a pair of crossbar arrays. The intermediate result of a **matrix-matrix multiplication (MatMul)** generated by one crossbar array can be directly written to another crossbar array, without using *analog-to-digital converters* (ADCs). Then, these cascaded crossbar arrays are used to perform subsequent analog MVMs. In this way, ReCAT can hide the ReRAM write latency and eliminate massive unnecessary AD conversions for intermediate results.
- We design a data mapping scheme to store signed operands in cascaded crossbar arrays based on offset binary, and thus these signed operands can be processed with analog MVMs correctly. For a pair of cascaded crossbar arrays, we encode both input and weight matrices with offset binary for the first crossbar array, and then directly write the intermediate results of a MatMul operation (multiple MVMs) to the second crossbar array. When the following MVM operation is conducted by the second crossbar array, we extract the real results by subtracting offset terms via a few periphery circuits.
- We propose an ADC virtualization scheme to share scarce ADCs among a group of crossbar arrays. Unlike traditional ReRAM-based PIM architectures that an ADC is tightly coupled with a crossbar array, ReCAT allows ADCs to be multiplexed by a group of crossbar arrays via a **time division multiplexing (TDM)** mechanism. In this way, a single crossbar array can harvest more ADCs for an MVM operation, and an ADC can be used by a group of crossbar arrays. Thus, when intermediate results are written into a crossbar array, the decoupled ADCs can be used by its neighboring crossbar arrays. This ADC virtualization scheme can significantly improve the utilization of ADCs to eliminate the performance bottleneck of MVM operations.

We prototype ReCAT and simulate different Transformer networks in MHSim [29]. We evaluate the inference performance and the energy efficiency for different Transformer models, and compare ReCAT with two state-of-the-art ReRAM-based PIM architectures designed for Transformer networks—ReTransformer [50] and ReBert [23]. Experimental results show that ReCAT achieves 3.06× and 2.11× performance speedup compared with ReTransformer and ReBert, respectively. ReCAT also reduces 31.47% and 24.51% of total energy consumption on average compared with ReTransformer and ReBert, respectively.

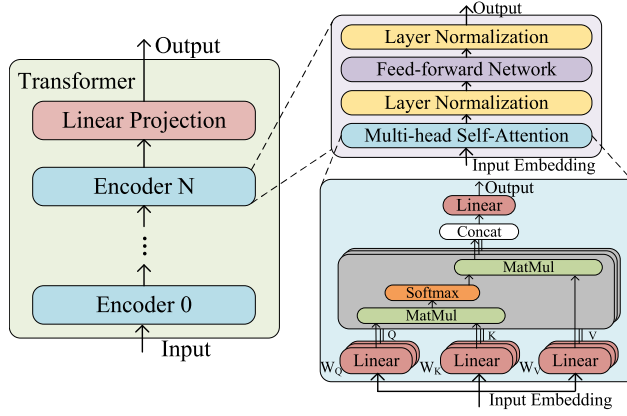


Fig. 1. The network structure of transformer.

2 Background and Motivation

2.1 Transformer Neural Networks

As shown in Figure 1, the Transformer model often consists of multiple encoder blocks. Each encoder commonly consists of a **feed-forward network (FFN)** layer, a **multi-head self-attention (MSA)** layer, and **layer normalization (LN)** blocks. An FFN layer contains two **fully-connected (FC)** layers, which can be modeled as matrix multiplication operations. The MSA layer is defined as follows:

$$\begin{aligned}
 Q_i, K_i, V_i &= X \cdot W_q^i, X \cdot W_k^i, X \cdot W_v^i \\
 \text{Head}_i(X) &= \text{Softmax} \left(\frac{Q_i \cdot K_i^T}{\sqrt{d_k}} \right) \cdot V_i \\
 \text{MSA}(X) &= \text{Concat}(\text{Head}_i(X)) \cdot W_o
 \end{aligned} \tag{1}$$

where $X \in \mathbb{R}^{n \times d}$ represents n input tokens (vectors) with a dimension of d . $W_q^i, W_k^i, W_v^i \in \mathbb{R}^{d \times d_k}$ are trainable weight matrices, where i denotes the index of heads (h) and $d_k = d/h$ is the embedding dimension. Q_i, K_i , and V_i denote the query vectors, key vectors, and value vectors, respectively. $W_o \in \mathbb{R}^{h \cdot d_k \times d}$ is the output weight matrix of a multi-head self-attention layer. The *Concat* operation concatenates results of multi-head self-attentions. The *Softmax* operation can be formulated by Equation (2).

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^{d_k} e^{x_j}} \tag{2}$$

According to Equation (1), the MSA layer involves massive time-consuming matrix-matrix multiplications and softmax operations. To better illustrate the proportion of each operation in MSA layers, we profile the execution time of different operations with six pre-trained models (BERT [10], BART [24], RoBERTa [30], ViT [11], DeiT [41], LeViT [14]) from HuggingFace [44]. BERT, BART, and RoBERTa models are used for question answering with SQuADv1 [33], while ViT, DeiT, and LeViT models are used for image classification with ImageNet-1k [34]. We measure the execution time of model inference on a Tesla V100 GPU. As shown in Figure 2, the projection of Q_i, K_i , and V_i (i.e., QKV_proj) usually spends more than 60% of the total execution time for all models except LeViT. The multiplication between the concatenate of multiple heads and W_o (i.e., SA_out) also consumes about 20% of total execution time in MSA layers. These experimental results imply that

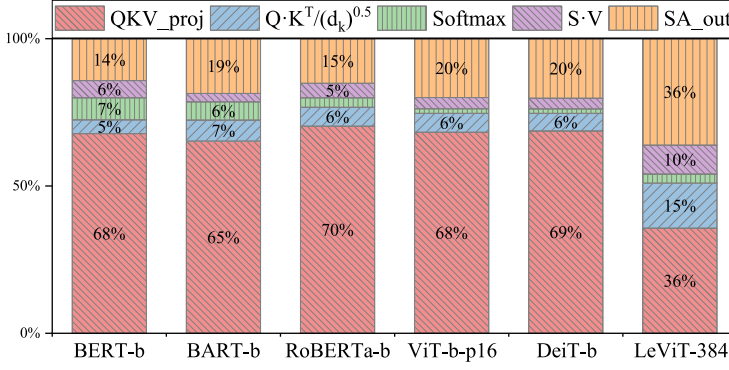


Fig. 2. Breakdown of the execution time for different transformers on GPUs.

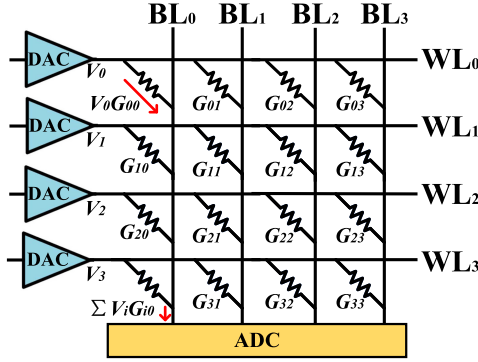


Fig. 3. A 4x4 ReRAM crossbar structure.

the matrix-matrix multiplication for the Q, K, V and the output embedding are the performance bottleneck of MSA layers during the Transformer inference.

2.2 ReRAM Preliminaries

Resistive random access memory (ReRAM) is an emerging non-volatile memory technology that can encode (store) data with a resistance. Multiple ReRAM cells can form a crossbar array that is able to perform *in-situ matrix-vector multiplication* (MVM) operations. As shown in Figure 3, each ReRAM cell represented with a conductance G_{ij} is placed at each cross-point between **wordlines** (WLs) and **bitlines** (BLs). By applying a voltage (V_i) to each WL, the output currents are accumulated in each BL according to Kirchhoff's circuit laws, i.e., the output of each BL is $\sum_i V_i \cdot G_{ij}$. To perform an *in-situ* MVM, digital-to-analog converters (DACs) are employed to convert digital input vectors to analog voltages. At the end of BL, *analog-to-digital converters* (ADCs) are used to convert dot-product results into digital values. In this way, the time complexity of an MVM operation is $O(1)$. This ReRAM-based crossbar structure has been demonstrated as a promising solution of *processing-in-memory* (PIM) for matrix-matrix multiplication acceleration in many previous studies [1, 3, 6, 8, 23, 25, 31, 35, 37, 38, 43, 47, 49, 50, 53].

2.3 Motivations and Challenges

Since weight matrices are fixed during inference of CNNs/DNNs, they often are stored in ReRAM crossbar arrays in advance to avoid the costly weight mapping operation [1, 6, 35, 37, 53]. Thus,

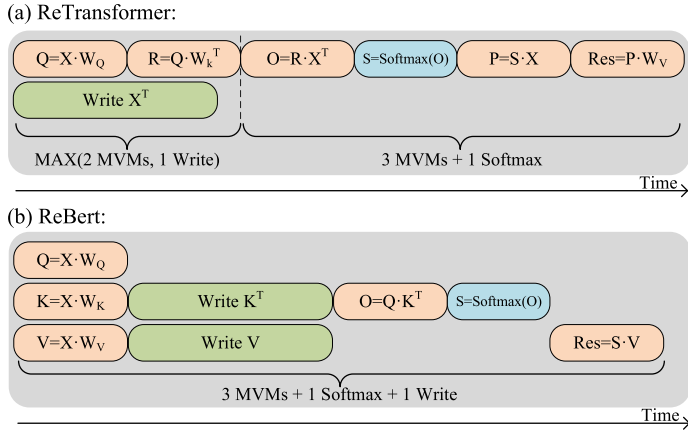


Fig. 4. The process flow of the self-attention in ReBert and ReTransformer.

the latency and energy consumption of mapping weight matrices can be ignored during the inference. However, in more complex Transformers, the Q, K, V and softmax results are generated dynamically and should be used in the following MVM operations, as shown in Equation (1). Thus, it's unfeasible to pre-map intermediate results into ReRAM crossbar arrays in advance, and the write latency and energy consumption for these intermediate matrices become non-trivial [42, 48, 50]. More importantly, these intermediate matrices also bring more AD/DA conversions, while the AD conversion is a performance bottleneck in ReRAM-based PIM architectures [5, 13, 15, 27, 43, 49, 53]. Since ReRAM crossbar arrays assigned for intermediate matrices are stalled during weight mapping, their peripheral circuits (e.g., ADCs) would be underutilized and eventually degrade the application performance.

To process these intermediate matrices in ReRAM-based PIM architectures, ReBert [23] maps the matrices K and Q to ReRAM crossbar arrays in the critical path, suffering from high latency of ReRAM writes and costly AD/DA conversions. ATT [16] customizes matrix-matrix multiplication circuits for these matrix multiplications in the digital domain to avoid writing K and V to ReRAM. However, according to Figure 2, the execution time of these matrix multiplications (i.e., $\frac{Q \cdot K^T}{\sqrt{d_k}}$ and $S \cdot V$) accounts for 7%-25% of the total execution time. Thus, these dedicated circuits may lead to more area overheads and performance degradation. ReTransformer [50] changes the process flow of self-attention based on matrix decomposition to reduce ReRAM write operations of intermediate results. Unlike classic Transformer models that should store intermediate matrices (K and V), ReTransformer overlaps the write operation of matrix X^T with other computations of Q and R , as shown in Figure 4(a). However, ReTransformer has to serialize all MVM computations within an MSA layer due to data dependency. The total execution time of an MSA layer becomes $(3 \text{ MVMs} + \text{MAX}(1 \text{ write}, 2 \text{ MVMs}) + 1 \text{ softmax})$. In contrast, ReBert [23] parallelizes write operations of K^T and V , and thus the total execution time is $(3 \text{ MVMs} + 1 \text{ write} + 1 \text{ softmax})$, as shown in Figure 4(b). Yang et al. [48] exploit analog signal memory and multiply-add circuits to perform matrix multiplications, without DA/AD conversions. However, since several parameters (e.g., the $\sqrt{d_k}$) of the Transformer network are fixed, the corresponding peripheral circuits also should be customized, lacking scalability to adapt to various Transformer networks. Overall, previous proposals have not efficiently handled these intermediate matrices generated in MSA layers.

To efficiently write intermediate matrices into ReRAM crossbar arrays, an optional approach is to use transimpedance amplifiers (TIAs) [8] to cascade two crossbar arrays. TIAs can convert

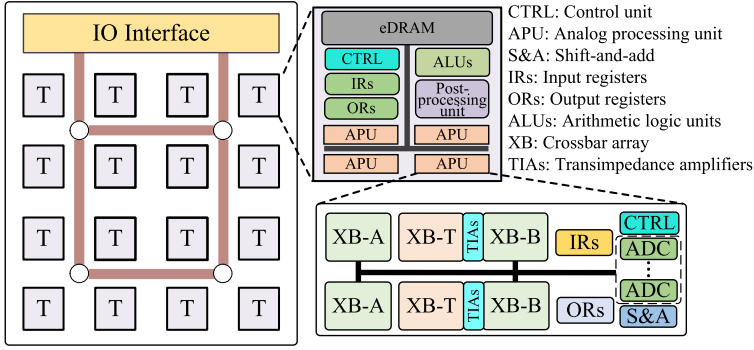


Fig. 5. An overview of ReCAT.

the output currents of one crossbar array into a voltage vector, which is then applied to another crossbar array to store the output result of an MVM operation in a column or a row, without involving in time-consuming AD conversions. In this way, the MVM operation on one crossbar array and data storing on another crossbar array can be performed in a pipeline, and thus the long latency of ReRAM writes is hidden. However, this crossbar architecture poses two major challenges for *in-situ* computing.

How to handle negative operands for cascaded crossbar arrays? A few proposals use double (i.e., positive/negative) crossbar arrays [6, 8] to process signed operands. Signed inputs are divided into positive and negative portions, and then mapped to two crossbar arrays to perform the MVM operation separately. However, this approach usually results in low storage efficiency and doubles the processing latency of an MVM operation. Some studies exploit complement and bias codes [35] to handle signed operands in ReRAM crossbar arrays. In our cascaded crossbar architecture, the accumulated currents in the second crossbar array should subtract different offsets that correspond to the counts of ‘ones’ in the output of the analog MVM operation. However, the computational complexity of calculating these offsets is almost equivalent to that of an MVM operation. Therefore, this approach using complement and bias codes is also inefficient for our cascaded crossbar architecture.

How to efficiently utilize the ADC resource in ReRAM crossbar arrays? In traditional ReRAM-based PIM architectures, very few (usually one) ADCs are tightly coupled with one crossbar array and shared by all bitlines. Thus, when the intermediate results are written into a crossbar array, the corresponding ADCs are unused, lowering the utilization of ADC resource. Since ADCs are the performance bottleneck of ReRAM-based PIM accelerators, they usually have a significant impact on the performance of accelerators. Thus, it is essential to design a flexible ADC sharing scheme to improve the utilization of ADC resource.

3 Design

In this section, we first introduce the architecture of ReCAT, including the cascaded crossbar arrays and the ADC sharing scheme. Then, we describe the data mapping scheme for signed operands and the allocation of ADC resource.

3.1 Overview

Figure 5 shows an overview of the ReCAT architecture. A ReCAT chip contains a global I/O interface to access the main memory, an on-chip controller, and multiple tiles. All tiles are connected with a concentrated mesh. Each tile contains an eDRAM buffer, an **output register**

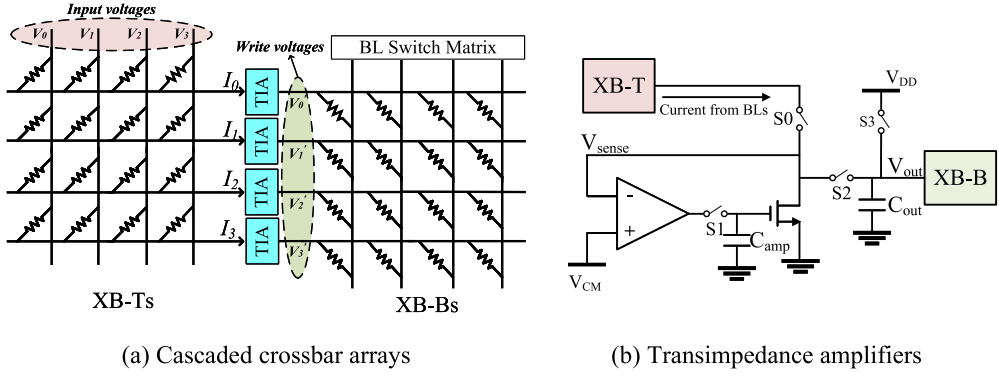


Fig. 6. The cascaded structure of two crossbar arrays.

(OR), a controller, multiple analog processing units, multiple *arithmetic logic units* (ALUs), and a post-processing unit. Each analog processing unit consists of 24 crossbar arrays (16 XB-As and 8 XB-Ts), 32 buffer arrays (XB-Bs), 16 ADCs, 16 shift-and-adders, 16 *analog-to-digital converters* (ADCs), and several **input/output registers** (IRs/ORs). XB-As and XB-Ts represent crossbar arrays that are directly connected with ADCs and *transimpedance amplifiers* (TIAs). The post-processing unit consists of an **activation unit** (AU) for non-linear functions, a **softmax unit** (SU) for softmax functions, and a **normalization unit** (NU) for layer normalization functions.

3.2 Cascaded Crossbar Array

In our cascaded crossbar architecture, *transimpedance amplifiers* (TIAs) [8] are used to cascade XB-Ts and XB-Bs so that the output of the XB-T can be directly written into the XB-B. As shown in Figure 6(a), the accumulated currents in BLs of an XB-T can be converted into voltages which are then used as write voltages for the XB-B. After the write operation, each ReRAM cell in the XB-B stores a conductance, whose value represents the accumulated current generated by the XB-T. In this way, we can store all intermediate outputs in XB-Bs, without involving in any AD and DA conversions. At last, new input voltages can be applied to WLs of XB-Bs for subsequent MVM operations.

Figure 6(b) shows the structure of TIAs. The TIA converts a current into a voltage in three phases. First, it closes S0 and S1 to activate the operational transconductance amplifier, which converts the output current into a voltage on C_{amp} , while S2 is open and S3 is closed to precharge C_{out} . Second, it opens S0, S1, and S3, and closes S2, allowing the sampled voltage on C_{amp} to drive the transistor and reproduce the output current. In this phase, C_{out} is discharged. Finally, it opens S2, allowing C_{out} to apply its voltage to the XB-B.

In Transformers, three weight matrices (i.e., W_q , W_k , W_v) are mapped in ReRAM crossbar arrays. The W_q is mapped to XB-As, while W_k , W_v are mapped to two XB-Ts. Figure 7 shows the process flow of ReCAT. Input vectors are simultaneously applied to both XB-As and XB-Ts. After analog MVMs, output currents (i.e., Q) from XB-As (with W_q) are converted into digital operands through ADCs, while other currents (i.e., K^T and V) are converted into write voltages and directly written to buffer crossbar arrays (i.e., XB-Bs) simultaneously. After Q is read out and K is mapped, Q is used as the input matrix and is applied to XB-Bs that map K^T . Thus, analog MatMul results of $Q \times K^T$ are accumulated and converted into digital values. Similarly, XB-Bs with V are used to calculate the output of another head after the softmax operation. To calculate the results of an MSA layer from outputs of multiple heads, W_o is mapped in XB-As since the output will be read out directly via ADCs. Besides the MSA layer, since FFN layers only involve MVMs using fixed weight matrices, we also use XB-As to map these weight matrices.

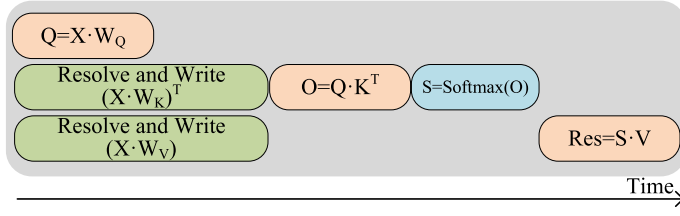


Fig. 7. The process flow of self-attention operations in ReCAT.

3.3 Handling Signed Operands

Since the analog partial sums are directly written into buffer crossbar arrays without using ADCs, the buffer crossbar arrays can only store positive sums. To correctly handle signed operands, one feasible solution is to use complement and bias codes [35]. However, it is difficult to resolve intermediate results in XB-Bs since each intermediate result contains a unique bias. Another solution [6, 8] is to use positive and negative arrays for signed partial sums. However, it would lower the storage efficiency (more than 50% zero values in XB-Bs on average).

ReCAT employs offset binary to encode signed operands in both input matrices and weight matrices for XB-Ts. After analog MVMs, the accumulated intermediate results in the form of offset binary are directly written to buffer arrays. To extract the actual value from the offset binary, each accumulated result should subtract a number that is correlated with both the input vector and the weight matrix. We note that the offset binary approach does not require additional storage bits because the fixed offset (128) does not cause an overflow error.

Assuming the offset value is b , the input matrix is $V = [v_{ij}]_{n \times m}$ and the weight matrix is $W = [w_{ij}]_{m \times l}$, the dot product of V and W (i.e., Z_{ij}) and its offset binary (Z'_{ij}) can be represented by Equation (3). For analog dot products, we can only get Z' and store it in the cascaded crossbar array. As shown in Equation (4), Z_{ij} can be figured out by subtracting $\sum_{k=0}^{m-1} w_{kj} \cdot b$ and $\sum_{k=0}^{m-1} (v_{ik} + b) \cdot b$ (we call them WB_j and VB_i in the following), where the first term can be pre-calculated and the latter term can be obtained by counting the number of 'ones' for each input vector with peripheral circuits.

$$Z_{ij} = \sum_{k=0}^{m-1} v_{ik} \cdot w_{kj} \quad (3)$$

$$Z'_{ij} = \sum_{k=0}^{m-1} ((v_{ik} + b) \cdot (w_{kj} + b))$$

$$Z_{ij} = Z'_{ij} - \sum_{k=0}^{m-1} w_{kj} \cdot b - \sum_{k=0}^{m-1} (v_{ik} + b) \cdot b \quad (4)$$

Although the offset binary for a given operand is fixed, we find that the offset ($WB_j + VB_i$) for each intermediate result is unique. This poses a challenge to get the real dot-product result from XB-Bs since the cost of resolving these unique offsets is almost comparable to the computational cost of an MVM. More specifically, for a given input vector (e.g., $X = [x_0, \dots, x_{n-1}]$), the accumulated result should subtract $\sum_{i=0}^{n-1} x_i \cdot (WB_j + VB_i)$, where i and j are row and column indices of the buffer matrix, respectively. This extra multiplication and accumulation for the real result lead to a new dot-product operation, which offsets the performance gain due to avoiding AD conversions.

Fortunately, for the output matrix of a MatMul, we find that elements in the same row share the same VB_i , and elements in the same column share the same WB_j , as shown in Figure 8. Therefore,

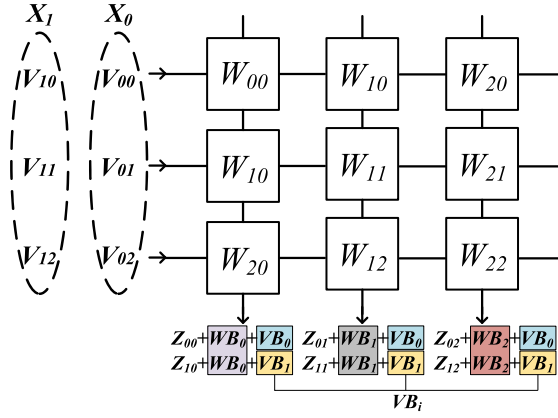


Fig. 8. The distribution of WB and VB when applying voltages to XB-Ts (The same color represents the same value).

we can still use the peripheral circuits [35] to sum up the input stream, and then multiply it with different WB_j to figure out $\sum_{i=0}^{n-1} x_i \cdot WB_j$. Since different columns of the intermediate matrix use the same VB_i , we can figure out $\sum_{i=0}^{n-1} x_i \cdot VB_i$ in the digital domain for all columns of the intermediate matrix. Hence, for each input vector, we only require one additional vector-vector dot-product operation, minimizing the performance degradation due to signed operands. To this end, we encode both inputs and weights using offset binary to calculate intermediate results in the analog domain, and exploit reasonable peripheral circuits to calibrate the final results.

3.4 Data Mapping Scheme

Similar to CASCADE [8], we use operands with a precision of 8-bit. As the precision of a ReRAM cell is limited, we employ the bit-slice scheme in CASCADE [8] to split each operand and map it into multiple ReRAM cells. We use four 64×64 XB-As to map a 64×64 matrix, where each cell in XB-As stores a 2-bit slice of an operand. Operands in weight matrices are encoded with an offset binary representation. Thus, both XB-A and XB-T use the same mapping scheme for weight matrices. However, the encoding schemes of their inputs are different. Specifically, inputs of XB-As are encoded with 2's complement, while inputs of XB-Ts are encoded with offset binary.

As we apply a multi-clock input scheme for multi-bit input operands [18], the XB-T would generate a vector of partial sums for every 2 bits of input operands. Therefore, for 8-bit input operands, each BL in the XB-T would generate 4 partial sums with different exponents to compose the output value. We use 4 XB-Bs to buffer these 4 partial sums. For 4 input slices and 4 weight slices, each operand in XB-Bs consists of 7 partial sums with different exponents. Thus, 7 AD conversions are required for a column of operands because partial sums with different exponents cannot be accumulated together in the analog domain. Since low-order bits have less contribution to the precision of the final result while incurring more ReRAM and ADC resource overheads, we follow ReBert [23] to truncate the last 3 low-order bits from the partial sums, and thus mitigate the performance overhead of AD conversions and save the ReRAM resource.

In our data mapping scheme of XB-Bs, multiple cells in the same buffer array have the same contribution to the precision of the intermediate result, and thus should be summed together to form the intermediate result. However, summing these partial sums in the digital domain requires extra AD conversions, which offsets the benefit of cascaded crossbar arrays. To avoid extra AD conversions, we employ the analog adder design [27] to accumulate currents in multiple

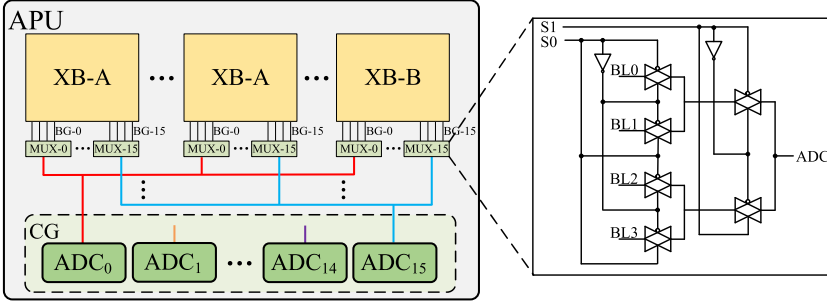


Fig. 9. The ADC sharing architecture in an analog processing unit.

BLs from different crossbar arrays. In this way, multiple XB-Bs can be deemed as a single large crossbar array. Since different XB-Bs in an APU share the same input vectors, we employ analog input buffers [27] for sharing input voltages among different XB-Bs to reduce the cost of WL drivers.

The encoding of inputs for XB-Ts is different from that for XB-Bs. As mentioned in Section 3.3, we use offset binary to encode inputs for XB-Ts, and then directly map the partial sums to XB-Bs. To simplify the processing of negative operands, we use the 2's complement representation to encode inputs for XB-Bs. Similar to ISAAC [35], we employ the counting peripheral circuits to sum up the input data. For example, in the cycle 0, we apply the first 2-bit in the input vector to XB-Bs and sum up them as c . Then, we can multiply c with each VB_i to figure out one offset. In this way, we can also figure out the offset in the cycle 1. Then, we shift left by 2 bits on the offset obtained in cycle 1, and sum up these two offsets. We repeat the above operations for each input bit and finally figure out the full offsets VO . We should change the sign of both the analog output and the offset since the MSB of inputs represents -2^7 in the 2's complement representation.

Beyond VO that can be generated from input vectors, another offset (WO) related to WB_j should be calculated from weight matrices. Since both WB_j and the input vector are split into 4 slices, we employ 32 ALUs (16 multipliers and 16 adders) to calculate the dot product of WO and an input vector with 64 elements in the digital domain during an analog MVM operation (required four cycles). In this way, we can resolve two offsets (i.e., VO and WO) in one MVM cycle and use them to extract the real results with signed operands.

3.5 ADC Resource Sharing

Previous works [8, 27, 35, 50] usually attach only one ADC to one crossbar array, and multiplex the ADC by multiple BLs. In these ADC-crossbar tightly coupled architectures, when intermediate results (i.e., K and V) are being written to ReRAM crossbar arrays, the attached ADCs are under-utilized. Since the AD conversion is the performance bottleneck in ReRAM-based PIM accelerators [8, 15, 27, 49], we design an ADC virtualization scheme to share the ADC resource among a group of crossbar arrays. The primary goal is to enable each crossbar array to utilize multiple ADCs for an MVM operation, and an ADC can be used by multiple crossbar arrays via *time division multiplexing* (TDM). Since the proposed ADC virtualization design allows crossbar arrays in an APU to utilize ADCs more efficiently, it is also applicable for other read circuits that have a larger area than that of a crossbar array, such as time-to-digital converters [27].

Figure 9 shows the ADC sharing architecture in an APU. All 16 ADCs are clustered as a **converter group (CG)** for all crossbar arrays. CG receives 16 analog inputs from crossbar arrays and generates 16 digital outputs in each cycle. Since we connect multiple BLs across 4 XB-Bs through 4×64 analog adders, 32 XB-Bs can be regarded as 8 general crossbar arrays (XB-As). To

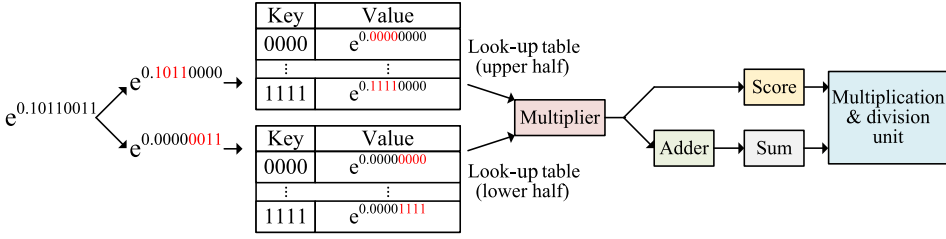


Fig. 10. The structure of the softmax processing unit.

map 16 ADCs in a CG to these BLs, we use a mapping structure similar to the direct mapping for caches. As shown in Figure 9, we first group every 4 consecutive BLs in each crossbar array through 4-to-1 multiplexers. Each 4-to-1 multiplexer is composed of 6 transmission gates. It receives 4 input currents from 4 BLs and routes one of these currents to the ADC using two control signals (i.e., S0 and S1 in Figure 9). Different MUXes within a crossbar array share the same control signals to simplify the control flow. All 16 BL groups (BGs) in a 64×64 crossbar array are indexed in ascending order and connected to different ADCs. In this way, each crossbar array can exploit all ADCs in an APU. To multiplex the CG with multiple crossbar arrays, we group BGs in different crossbar arrays with the same index and connect them to one ADC. For example, all BGs with index 0 are connected to ADC₀. Thus, each ADC in the CG is multiplexed by 16 crossbar arrays. Therefore, all ADCs are fully utilized even when only one crossbar array in an APU is activated. We note that this architecture only incurs a minor modification to the connection between multiplexers and ADCs, while crossbar arrays and other periphery circuits remain the same as previous works [1, 31, 35, 53].

Since we decouple the mapping between ADCs and crossbar arrays, the ADC resource can be utilized more efficiently. To accelerate the self-attention operation, we first conduct analog MVMs for $X \cdot W_K$ and $X \cdot W_V$ in XB-Ts and directly map partial sums of K^T and V into XB-Bs. During data mapping, analog MVMs of $Q \cdot K^T$ are stalled because Q and K have not been completely calculated yet. Therefore, we assign all ADCs in the APU to XB-As to convert the analog outputs of $X \cdot W_Q$ simultaneously. In this way, the analog output Q can be converted into digital values more rapidly, while K is directly mapped to buffer arrays without AD conversions. After Q is generated and K is mapped, XB-Bs start to perform $Q \cdot K^T$, and all ADCs are utilized by XB-Bs to convert the analog output. With the assistance of peripheral circuits (e.g., ALUs), the output of $Q \cdot K^T$ with signed operands is generated and sent to softmax processing units for softmax operations.

To efficiently process the softmax function in ReCAT, we employ the design of softmax processing unit [17, 51] that consists of two look-up tables for exponential functions. Figure 10 shows the structure of the softmax processing unit. Two look-up tables are used to store the upper and lower halves of the exponential function, respectively. By multiplying the results of two look-up tables, the softmax processing unit gets the output of the exponential function. Then, adders, registers, and multiplication-and-division units are used for subsequent operations in the softmax function.

According to Equation (1), the result of softmax should be multiplied with V to generate the final result of an attention operation. Similar to K^T , V is simultaneously mapped to XB-Bs without using any ADCs. When the softmax operation is finished, $S \cdot V$ can be conducted in XB-Bs immediately. We also assign all ADCs to these XB-Bs to accelerate AD conversions. We repeat the above operations to calculate the result of the multi-head attention, and finally generate the MSA result by multiplying another matrix- W_O . As W_O is pre-map in XB-As, the final matrix multiplication is also conducted with the CG efficiently. With this ADC virtualization scheme and dynamic ADC resource allocation, we can fully utilize the scarce ADCs for MSA layers.

3.6 Pipeline

In ReCAT, 8-bit 1.25 GHz ADCs [52] are used for AD conversions. With our ADC sharing scheme, an MVM operation for Q spends 51.2 ns. According to CASCADE [8], the write operation to a column or a row can be done within 25 ns. Since each cell in XB-As can only store a 2-bit slice of an 8-bit operand, 4×25 ns is required to write an output vector of K . Therefore, the latency of storing K can be partially overlapped with the latency of AD conversions for Q . At first, ReCAT can complete the computation of Q , K , and V . Then, ReCAT sequentially conducts the following operations in Equation (1) due to data dependency. However, since the bottleneck resource (i.e., ADCs) is fully utilized at any time through our ADC sharing scheme, the total execution time of the self-attention is significantly reduced compared with ReBert [23] and ReTransformer [50].

The process flow of ReBert is similar to ReCAT. However, since we can dynamically allocate the critical resource—ADCs, the latency of each MVM operation in ReCAT is less than that in ReBert when these two accelerators equip the same ADCs. Unlike ReTransformer which resolves Q , K , and V sequentially, ReCAT can simultaneously perform MVMs for these matrices. Thus, ReCAT also achieves higher performance than ReTransformer in processing an MSA layer.

Beyond MSA layers, ReCAT further employs an inter-layer pipeline to process multiple layers simultaneously. All ADCs in ReCAT can be leveraged simultaneously during inference by balancing the inter-layer pipeline. Therefore, ReCAT can realize the potential of ReRAM-based PIM accelerators by significantly improving ADC utilization.

4 Experiments

4.1 Methodology

We simulate the latency, area, and energy consumption of the cascaded ReRAM crossbar architecture for different Transformers at 32 nm technology based on an instruction-driven simulator—MHSim [29]. In our experiments, TaOx/HfOx [45] is adopted as the ReRAM device. To evaluate the inference accuracy of ReCAT, we also adopt the device behavioral models in NeuroSim [5] to simulate non-ideal properties of ReRAM devices such as the non-ideal weight update and read variation. To simulate the process variation of ReRAM devices, we adopt a simulation scheme [4] to model the process variation as a log-normal distribution with 0 mean and a standard deviation of σ . We set $\sigma = 0.05$ for a low process variation. We also adopt the circuit-level models in NeuroSim to simulate the latency and power of peripheral circuits associated with crossbar arrays. Based on the circuit-level models, we count each MVM operation to calculate the latency and energy consumption of peripheral circuits. Similar to CASCADE [8], we employ 1T1R ReRAM crossbar arrays and adopt its write scheme. Thus, the latency of writing a row or a column into a crossbar array is set to 25 ns. The area and energy consumption of the customized peripheral circuits (e.g., softmax unit) are simulated by Synopsys Design Compiler [9]. We use Booksim 2.0 [19] and Orion 3.0 [22] to simulate the latency and the energy consumption of the on-chip interconnection network, respectively. We use CACTI 7.0 [2] to estimate the area and energy consumption of all input/output buffers.

4.2 Experimental Setup

System Configurations. In our experiments, we use a physical server equipped with Intel Xeon E5-2650v3 CPU, 64 GB DDR-4 memory, and a Tesla V100 GPU as the GPU platform. Table 1 shows the specification of key components in a single chip of ReCAT. In total, 16 chips are used to construct a whole accelerator. We use the parameters of analog input buffers and analog adders according to TIMELY [27], and adopt the TIA design in CASCADE [8] to cascade crossbar arrays. All operands [21] are encoded with 8-bit signed fixed-point numbers. We use 8-bit 1.25 GHz ADCs [52] to convert analog signals into digital values. Like most ReRAM-based PIM accelerators, we use

Table 1. ReCAT Specification

Component	Configuration
XB-A	Size: 64×64, Number: 16/APU, Power: 1.2 mW, Area: 0.0008 mm ²
XB-T	Size: 64×64, Number: 8/APU, Power: 0.6 mW, Area: 0.0004 mm ²
XB-B	Size: 64×64, Number: 32/APU, Power: 2.4 mW, Area: 0.0016 mm ²
ADC	Resolution: 8-bit, Number: 16/APU, Power: 30.4 mW, Area: 0.005 mm ²
DAC	Resolution: 2-bit, Number: 64/APU, Power: 0.25 mW, Area: 0.00001 mm ²
TIA	Number: 8×64/APU, Power: 0.004 mW, Area: 0.00008 mm ²
MUX	Number: 384/APU, Power: 0.1 mW, Area: 0.00001 mm ²
IR (APU)	Size: 2 KB, Number: 1/APU, Power: 1.24 mW, Area: 0.0021 mm ²
OR (APU)	Size: 3 KB, Number: 1/APU, Power: 1.68 mW, Area: 0.0032 mm ²
Controller (APU)	Number: 1/APU, Power: 0.382 mW, Area: 0.0015 mm ²
APU	Number: 8/tile, Power: 306.05 mW, Area: 0.118 mm²
ALU	Number: 64/tile, Power: 3.2 mW, Area: 0.0038 mm ²
eDRAM	Size: 64 KB, Number: 1/tile, Power: 20.7 mW, Area: 0.083 mm ²
SU	Size: 512 B, Number: 1/tile, Power: 1.134 mW, Area: 0.0072 mm ²
IR (Tile)	Size: 2 KB, Number: 1/tile, Power: 1.24 mW, Area: 0.0021 mm ²
OR (Tile)	Size: 3 KB, Number: 1/tile, Power: 1.68 mW, Area: 0.0032 mm ²
AU & NU	Number: 1/tile, Power: 0.575 mW, Area: 0.0012 mm ²
Controller (Tile)	Number: 1/tile, Power: 0.5 mW, Area: 0.00145 mm ²
Tile	Number: 128/chip, Power: 42.9 W, Area: 28.1, mm²
Router	Number: 32/chip, Power: 1.344 W, Area: 4.832 mm ²
HyperTransport	Bandwidth: 6.4GB/s, Number: 4/chip, Power: 10.4 W, Area: 22.88 mm ²
Chip Total	Power: 54.63 W, Area: 55.81 mm²

Table 2. Benchmarks

Model	Specifications	Dataset
BERT-b	12-MSA 13-FFN	SQuAD
BART-b	18-MSA 13-FFN	
RoBERTa-b	12-MSA 13-FFN	
ViT-b-p16	1-CONV 12-MSA 13-FFN	ImageNet-1k
DeiT-b	1-CONV 12-MSA 13-FFN	
LeViT-384	4-CONV 12-MSA 16-FFN 2-Downsampling MSA	

multi-level cells (2-bit) in XB-As and XB-Ts [35, 53], and employ 2-bit *digital-to-analog converters* (DACs) [18, 27, 43]. We employ the HyperTransport link model [27] for off-chip communication.

Benchmarks. We use typical pre-trained Transformer models—BERT [10], BART [24], RoBERTa [30], ViT [11], DeiT [41], and LeViT [14] to evaluate the inference accuracy, performance, and energy consumption of different architectures for different tasks. We use SQuADv1 [33] as a question-answering dataset, and use ImageNet-1k [34] as an image classification dataset. All pre-trained models are obtained from HuggingFace [44]. The model parameters are shown in Table 2.

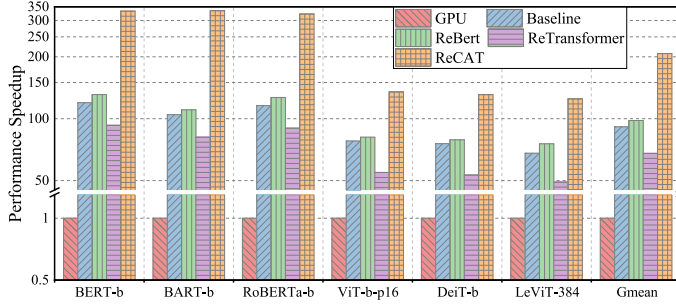


Fig. 11. Performance speedup of different architectures for various models, all normalized to GPU.

Alternatives for Comparison. To evaluate the effectiveness of ReCAT, we compare ReCAT with GPUs and three ReRAM-based PIM architectures as follows. In our experiments, we use the same configuration of ADCs for all PIM architectures since the AD conversion is the performance bottleneck of analog MVM operations.

- **Real GPUs.** We use NVIDIA Tesla V100 GPU and the PyTorch [32] framework to run different DNN models in a physical server, and measure their execution time.
- **Baseline.** For comparison, we model a conventional crossbar architecture in which ADCs and crossbar arrays are tightly coupled, and intermediate results generated from one array should be mapped into another array after an AD conversion and a DA conversion.
- **ReBert [23].** ReBert is a ReRAM-based PIM architecture designed for Transformers with special algorithm optimizations on sparse attention. ReBert uses the same data mapping scheme as the Baseline.
- **ReTransformer [50].** ReTransformer changes the computation model of MSA layers to minimize ReRAM write operations. For a fair comparison with Baseline, ReCAT, and ReBert, we implement look-up tables for softmax operations in ReTransformer with CMOS logic circuits instead of using ReRAM in-memory logic technology.

4.3 Inference Performance

Figure 11 shows the inference performance of different Transformers on GPU, the Baseline, ReTransformer, ReBert, and the proposed architecture—ReCAT. ReCAT achieves about 207.3 $\times$, 2.27 $\times$, 2.11 $\times$, and 3.06 $\times$ performance speedup on average compared with the GPU platform, the Baseline, ReBert, and ReTransformer, respectively. All ReRAM-based PIM architectures achieve considerable performance speedups over the GPU platform because of the high efficiency of *in-situ* analog MVMs using ReRAM crossbar arrays, while the execution time of Transformers in the GPU platform is dominated by MVMs in FFN layers and MSA layers.

ReCAT achieves 2.27 $\times$ performance speedup over the Baseline on average since ReCAT can perform *in-situ* MVMs and the data storing of intermediate results concurrently. This design can hide the write latency of mapping intermediate results in MSA layers. In this way, the inter-layer pipeline would not be stalled at each MSA layer. In addition, as we discussed in Section 3.5, ADCs attached to those crossbar arrays are idle while storing intermediate results. As AD conversions are the performance bottleneck of analog MVMs, ReCAT exploits an ADC sharing scheme to orchestrate these ADCs for high utilization, and thus achieves higher performance. Although ReBert can improve the performance with their algorithm-level optimizations compared with the Baseline, ReCAT can still achieve 2.11 $\times$ performance speedup over ReBert on average. Because ReBert only alleviates the data hazard issue, but still suffers from high overhead of AD conversions

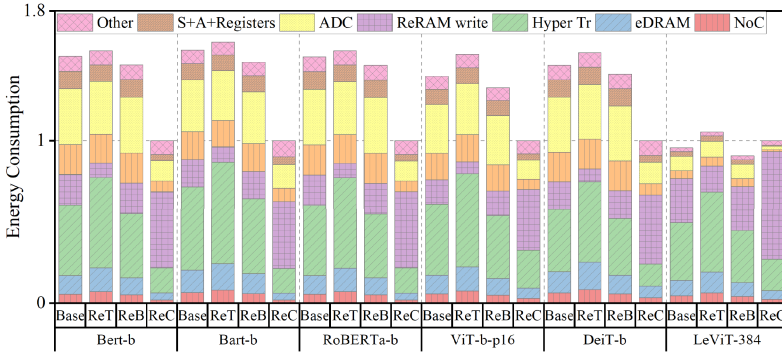


Fig. 12. Energy consumption of different architectures for different Transformer models, all normalized to ReCAT (“ReC”, “Base”, “ReT”, and “ReB” represent ReCAT, Baseline, ReTransformer, and ReBert, respectively).

and high write latency for intermediate results. In short, the performance speedup of ReCAT stems from the bypass of mapping intermediate results and the improvement of ADC utilization.

We find that ReTransformer shows less performance speedup than the Baseline and ReBert, because crossbar arrays in which X^T are stored can stall the pipeline of the process flow within the attention layer. Therefore, operations in MSA layers are executed sequentially, leading to a relatively low resource utilization. Although ReTransformer can hide the write latency of intermediate results, its computing model causes higher latency due to data dependency. Therefore, ReCAT can achieve $3.06\times$ performance speedup on average compared with ReTransformer.

4.4 Energy Consumption

Figure 12 shows the energy consumption of different architectures, all normalized to ReCAT. On average, ReCAT reduces total energy consumption by 27.72% and 24.51% compared with the Baseline and ReBert, respectively. The reason is that ReCAT eliminates AD conversions during storing intermediate results, and thus reduces energy consumption of ADCs, shift-adder, and registers. Besides these energy savings, ReCAT also reduces the energy consumption of HyperTransport by reducing the total execution time since HyperTransport has rather high leakage power consumption. Although ReCAT can reduce the energy consumption of the above components, it shows higher energy consumption when writing intermediate results to buffer arrays. For 2-bit DACs and 8-bit input vectors, XB-T generates 4 (i.e., $8/2$) partial sums with different orders, which should be stored in four different XB-Bs. Although ReCAT truncates low-order bits of intermediate partial sums from XB-Ts, more partial sums still lead to $1.5\times$ more write operations than mapping 8-bit digital operands. Thus, ReCAT incurs a relatively higher energy consumption for writing intermediate results compared with other architectures. For a lightweight Transformer model with a small number of MVM operations within its MSA layers (e.g., LeViT-384), the reduction of energy consumption due to reduced AD conversions is very limited. In this case, the energy consumption of ReRAM writes becomes dominant, causing lower energy efficiency compared with the Baseline and ReBert.

Compared with ReTransformer, ReCAT can reduce total energy consumption by 31.47% on average. Although ReTransformer achieves the least energy consumption of ReRAM writes among all PIM accelerators, the longer execution time results in more energy consumption for all benchmarks, and thus offsets the reduction of energy consumption for ReRAM writes.

These experimental results demonstrate that ReCAT can achieve higher energy efficiency compared with other PIM accelerators when it is applied to computing-intensive Transformer models.

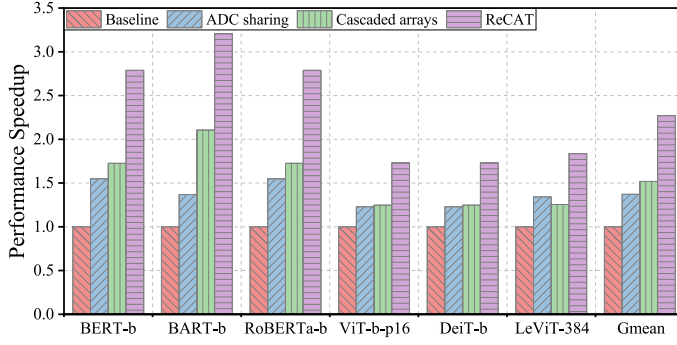


Fig. 13. Performance speedup with different technologies for different Transformers, all normalized to Baseline.

4.5 Effectiveness of Individual Technologies

Figure 13 shows the effectiveness of individual technologies proposed in ReCAT. The “ADC sharing” denotes that we only enable the ADC-crossbar decoupled structure for ADC sharing, and dynamically allocate idle ADCs to parallelize ReRAM write operations and analog MVM operations. The “cascaded arrays” denotes that we only enable cascaded crossbar architecture to store intermediate results of MatMul operations. As shown in Figure 13, both of the two technologies can achieve considerable performance improvement compared with the Baseline.

The “ADC sharing” scheme can fully utilize ADCs assigned to crossbar arrays that are being programmed for other computing tasks on XB-As, and thus improves the ADC utilization. Since ADCs are the performance bottleneck for analog computing, the application performance is significantly improved due to high ADC utilization. However, the unnecessary AD conversions for intermediate results still cause a waste of ADC resource. Therefore, the “ADC sharing” scheme only achieves 37% performance speedup on average over Baseline. With “cascaded arrays”, the write latency of intermediate results can be overlapped with other MVM operations. Thus, this design reduces pipeline stalls within each MSA layer. Moreover, it also eliminates the latency of AD conversions for intermediate results. However, since conventional ReRAM-based PIM architectures tightly couple ADCs and crossbar arrays, ADCs are underutilized when they are not active. Thus, the “cascaded arrays” design only leads to a limited performance speedup (52%) compared with Baseline. Putting these two technologies together, ReCAT can even improve the performance by $2.27\times$ on average compared with Baseline because they can collaborate to fully utilize ADCs and crossbar arrays.

4.6 Comparison of Different Data Mapping Schemes

In this section, we compare the performance of ReCAT with different data mapping (or encoding) schemes for signed operands. Two data mapping schemes proposed in ISAAC [35] and PRIME [6, 35] are commonly used in many ReRAM-based PIM architectures [1, 12, 37, 53]. We implement these two mapping schemes and evaluate their performance within the proposed architecture. “ReCAT_w/PRIME” uses positive and negative crossbar arrays to map positive operands and negative operands, respectively. “ReCAT_w/ISAAC” uses the offset binary for weights and 2’s complement representation for input vectors.

Figure 14 shows the performance of different data mapping schemes. “ReCAT_w/PRIME” shows lower performance than Baseline because the separate mapping of negative and positive weights doubles the resource requirement of crossbar arrays. For the same amount of array resource, “ReCAT_w/PRIME” can only utilize half of ReRAM arrays for *in-situ* computing. Moreover,

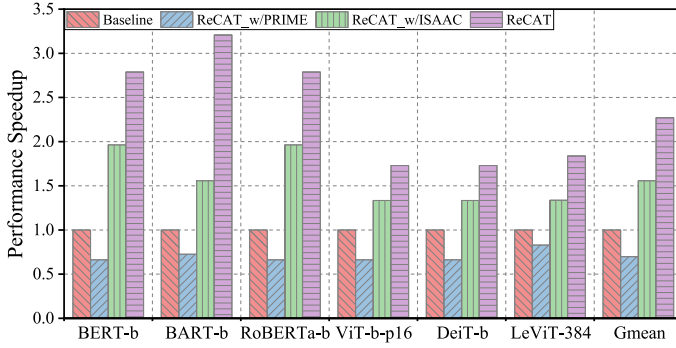


Fig. 14. Performance speedup of different data mapping scheme for different Transformers, all normalized to Baseline.

Table 3. Inference Accuracy of Different Benchmarks

	BERT-b	BART-b	RoBERTa-b	ViT-b-p16	DeiT-b	LeViT-384
GPU	84.60%	93.60%	89.60%	75.20%	82.81%	81.18%
Baseline	75.12%	83.20%	80.30%	71.76%	79.53%	75.69%
ReBert	75.05%	82.90%	80.10%	71.20%	78.74%	75.40%
ReTransformer	75.22%	83.15%	80.30%	71.76%	79.52%	75.72%
ReCAT	75.77%	84.38%	79.00%	69.02%	77.48%	74.90%

“ReCAT_w/PRIME” requires individual inputs for positive and negative operands. This ultimately doubles the number of *in-situ* MVM operations, and thus degrades the performance.

“ReCAT_w/ISAAC” shows reasonable performance improvement compared with Baseline. “ReCAT_w/ISAAC” requires a portion of buffer arrays to store negative partial sums. As *in-situ* analog computing can only conduct multiply-and-add operations, “ReCAT_w/ISAAC” requires an additional round of AD conversions to resolve the negative part of the final result. Thus, “ReCAT_w/ISAAC” requires twice the execution time to resolve the negative operands in XB-Bs. Overall, “ReCAT_w/ISAAC” exhibits an average performance speedup of 56% compared with Baseline. ReCAT uses a data mapping scheme similar to “ReCAT_w/ISAAC”, but outperforms the performance of “ReCAT_w/ISAAC” by 1.45× because ReCAT encodes both input vectors and weights with an offset binary representation, and thus avoids additional operations for handling negative values from XB-Ts. In contrast, “ReCAT_w/ISAAC” suffers costly AD conversions to handle positive and negative values generated by XB-Ts. These experiments demonstrate that the data mapping scheme of ReCAT can efficiently handle negative operands without involving in AD conversions, and thus significantly improves the performance compared with “ReCAT_w/ISAAC”.

4.7 Inference Accuracy

As we use buffer arrays to store intermediate results in the analog domain, the computation accuracy would be more sensitive to the non-ideal properties of ReRAM cells. We use MHSim to simulate non-ideal properties, including the write variation, the process variation, the read variation, and the non-linearity of ReRAM cells. Then we evaluate the impact of our proposal on the inference accuracy for different Transformer models.

Table 3 shows the inference accuracy of GPU, Baseline, ReBert, ReTransformer, and ReCAT for different Transformer models. We use the GPU platform to evaluate the inference accuracy that

can be achieved by software, without incurring computational errors. For BERT-b, BART-b, and RoBERTa-b, all ReRAM-based PIM architectures exhibit about 10% accuracy degradation. This is because the question-answering task of these models is sensitive to the computation accuracy of MVMs. For models used for the image classification such as ViT-b-p16, DeiT-b, and LeViT-384, all ReRAM-based PIM architectures show 3.28%-6.28% accuracy degradation. The accuracy degradation of image classification tasks is lower than that of question-answering tasks because the former can tolerate moderate computation errors in vision transformer models.

Among these ReRAM-based PIM architectures, Baseline and ReTransformer exhibit similar inference accuracy since analog intermediate results are converted into digital values through ADCs, and then processed in the digital domain. ReBERT shows a little accuracy degradation (<1%) compared with the Baseline because the proposed algorithm optimizations omit several computations. ReCAT shows only 1.15% accuracy degradation compared with the Baseline because ReCAT truncates low-order bits of intermediate partial sums from XB-Ts to save ReRAM resource. The truncation leads to a slight error during analog computing, and eventually affects the end-to-end inference accuracy of Transformer models slightly.

5 Related Work

We present the related work in the following categories.

Redesign Peripheral Circuits for ReRAM Crossbar Arrays. Several ReRAM-based PIM accelerators [8, 13, 40, 48] attempt to reduce AD conversions during analog MVM operations in neural networks. CASCADE [8] exploits TIAs to convert accumulated currents into write voltages, which then are applied to cascaded buffer arrays for multiply-and-add operations. Yang, et al. [48] design analog memory to store analog intermediate results instead of converting them into digital values. These analog values are directly used for subsequent processing without AD/DA conversions. This design requires massive analog multiply-add, analog softmax, and layer normalization circuits, and thus may degrade the performance and accuracy of analog computing. Sun et al. [40] also cascades multiple crossbar arrays to process neural networks in the analog domain. However, this architecture is designed for a specific neural network (i.e., VGGNet [36]), lacking scalability for other scenarios. RFSM [13] proposes a switch-matrix based structure for multiple crossbar arrays to perform convolution operations among multiple convolution layers without AD conversions. However, RFSM does not consider signed operands and lacks implementation details of switch matrices. Inspired by these designs, ReCAT also exploits TIAs to cascade a pair of crossbar arrays without using ADCs, and can effectively hide ReRAM write latency of intermediate results generated in Transformer layers by overlapping ReRAM writes and MVM operations.

ReRAM-based Accelerators for Transformers. ReBERT [23] proposes an algorithm-level optimization mechanism—window self-attention to reduce the attention computation scope. ReBERT can partially reduce pipeline stalls for storing intermediate results. ReTransformer [50] changes the computation model of MSA layers to minimize write operations. However, the total processing latency of an inference task is still high due to the data dependency and the resource contention of crossbar arrays. CPSAA [26] further optimizes the computation model of ReTransformer and exploits *ReRAM-based content addressable memory (ReCAM)* to accelerate sparse MVMs. However, CPSAA still suffers from costly AD conversions. ATT [16] uses CMOS-based processing units to avoid ReRAM write operations for intermediate MVMs. However, the relatively low computation parallelism usually degrades the performance of inference. Similar to ATT [16], X-Former [39] also employs dedicated CMOS-based processing units with SRAM-based crossbar arrays to handle intermediate results of Transformers. Most existing ReRAM-based PIM accelerators designed for Transformers still suffer from pipeline stalls because costly ReRAM write operations cannot be overlapped with other computations completely. ReCAT differs from previous proposals

in two-fold. First, ReCAT exploits cascaded crossbar arrays to mitigate the impact of writing intermediate results in Transformer and also avoid costly AD conversions. Second, ReCAT exploits an ADC virtualization scheme to further improve the utilization of the scarce ADC resource.

6 Conclusion

In this paper, we propose a ReRAM-based PIM architecture called ReCAT for Transformers. ReCAT exploits cascaded crossbar arrays to store intermediate results generated during attention operations in MSA layers, without costly AD conversions and stalling inter-layer pipelines of Transformers. In addition, ReCAT designs an ADC-crossbar decoupled structure associated with a dynamic resource allocation scheme to fully utilize the ADC resource. Experimental results show that ReCAT can achieve 207.3 $\times$, 2.11 $\times$, and 3.06 $\times$ speedup on average compared with state-of-the-art GPU, ReBert, and ReTransformer, respectively.

References

- [1] Aayush Ankit, Izzat El Hajj, Sai Rahul Chalamalasetti, Geoffrey Ndu, Martin Foltin, R. Stanley Williams, Paolo Faraboschi, Wen-mei W Hwu, John Paul Strachan, Kaushik Roy, and Dejan S. Milojevic. 2019. PUMA: A programmable ultra-efficient memristor-based accelerator for machine learning inference. In *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 715–731.
- [2] Rajeev Balasubramonian, Andrew B. Kahng, Naveen Muralimanohar, Ali Shafiee, and Vaishnav Srinivas. 2017. CACTI 7: New tools for interconnect exploration in innovative off-chip memories. *ACM Transactions on Architecture and Code Optimization* 14, 2, Article 14 (2017), 25 pages.
- [3] Nagadastagiri Challapalle, Sahithi Rampalli, Linghao Song, Nandhini Chandramoorthy, Karthik Swaminathan, John Sampson, Yiran Chen, and Vijaykrishnan Narayanan. 2020. GaaS-X: Graph analytics accelerator supporting sparse data representation using crossbar architectures. In *Proceedings of the ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. 433–445.
- [4] Gouranga Charan, Jubin Hazra, Karsten Beckmann, Xiacong Du, Gokul Krishnan, Rajiv V. Joshi, Nathaniel C. Cady, and Yu Cao. 2020. Accurate inference with inaccurate RRAM devices: Statistical data, model transfer, and on-line adaptation. In *Proceedings of the 2020 57th ACM/IEEE Design Automation Conference (DAC)*. 1–6.
- [5] Pai-Yu Chen, Xiaochen Peng, and Shimeng Yu. 2018. NeuroSim: A circuit-level macro model for benchmarking neuro-inspired architectures in online learning. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 37, 12 (2018), 3067–3080.
- [6] Ping Chi, Shuangchen Li, Cong Xu, Tao Zhang, Jishen Zhao, Yongpan Liu, Yu Wang, and Yuan Xie. 2016. PRIME: A novel processing-in-memory architecture for neural network computation in ReRAM-based main memory. In *Proceedings of the 43rd International Symposium on Computer Architecture (ISCA)*. 27–39.
- [7] Ye Chi, Jianhui Yue, Xiaofei Liao, Haikun Liu, and Hai Jin. 2024. A hybrid memory architecture supporting fine-grained data migration. *Frontiers of Computer Science* 18, 2 (2024), 182103.
- [8] Teyuh Chou, Wei Tang, Jacob Botimer, and Zhengya Zhang. 2019. CASCADE: Connecting RRAMs to extend analog dataflow in an end-to-end in-memory processing paradigm. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 114–125.
- [9] Synopsys compiler. 2023. <https://www.synopsys.com/implementation-and-signoff/rtl-synthesis-test/dc-ultra.html>. (2023).
- [10] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Association for Computational Linguistics, 4171–4186.
- [11] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2021. An image is worth 16 $\times$ 16 words: Transformers for image recognition at scale. In *Proceedings of the 9th International Conference on Learning Representations (ICLR)*. 1–21.
- [12] Ben Feinberg, Uday Kumar Reddy Vengalam, Nathan Whitehair, Shibo Wang, and Engin Ipek. 2018. Enabling scientific computing on memristive accelerators. In *Proceedings of the Forty-Fifth Annual International Symposium on Computer Architecture (ISCA)*. 367–382.

- [13] Yingxun Fu, Xun Liu, Jiwu Shu, Zhirong Shen, Shiye Zhang, Jun Wu, and Li Ma. 2021. Receptive-field and switch-matrices based ReRAM accelerator with low digital-analog conversion for CNNs. In *Proceedings of the 2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 244–247.
- [14] Ben Graham, Alaaeldin El-Nouby, Hugo Touvron, Pierre Stock, Armand Joulin, Hervé Jégou, and Matthijs Douze. 2021. LeViT: A vision transformer in ConvNet’s clothing for faster inference. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 12259–12269.
- [15] Peng Gu, Boxun Li, Tianqi Tang, Shimeng Yu, Yu Cao, Yu Wang, and Huazhong Yang. 2015. Technological exploration of RRAM crossbar array for matrix-vector multiplication. In *Proceedings of the 20th Asia and South Pacific Design Automation Conference*. 106–111.
- [16] Haoqiang Guo, Lu Peng, Jian Zhang, Qing Chen, and Travis D. LeCompte. 2020. ATT: A fault-tolerant ReRAM accelerator for attention-based neural networks. In *Proceedings of the 2020 IEEE 38th International Conference on Computer Design (ICCD)*. 213–221.
- [17] Tae Jun Ham, Sung Jun Jung, Seonghak Kim, Young H. Oh, Yeonhong Park, Yoonho Song, Jung-Hun Park, Sanghee Lee, Kyoung Park, Jae W. Lee, and Deog-Kyoon Jeong. 2020. A³: Accelerating attention mechanisms in neural networks with approximation. In *Proceedings of 2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 328–341.
- [18] Yu Huang, Long Zheng, Pengcheng Yao, Qinggang Wang, Xiaofei Liao, Hai Jin, and Jingling Xue. 2022. Accelerating graph convolutional networks using crossbar-based processing-in-memory architectures. In *Proceedings of the 2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 1029–1042.
- [19] Nan Jiang, Daniel U. Becker, George Michelogiannakis, James Balfour, Brian Towles, D. E. Shaw, John Kim, and William J. Dally. 2013. A detailed and flexible cycle-accurate network-on-chip simulator. In *Proceedings of the 2013 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 86–96.
- [20] Hai Jin, Cong Liu, Haikun Liu, Ruikun Luo, Jiahong Xu, Fubing Mao, and Xiaofei Liao. 2022. ReHy: A ReRAM-based digital/analog hybrid PIM architecture for accelerating CNN training. *IEEE Transactions on Parallel Distributed Systems* 33, 11 (2022), 2872–2884.
- [21] Norman P. Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, Rick Boyle, Pierre-luc Cantin, Clifford Chao, Chris Clark, Jeremy Coriell, Mike Daley, Matt Dau, Jeffrey Dean, Ben Gelb, Tara Vazir Ghaemmaghami, Rajendra Gottipati, William Gulland, Robert Hagmann, C. Richard Ho, Doug Hogberg, John Hu, Robert Hundt, Dan Hurt, Julian Ibarz, Aaron Jaffey, Alek Jaworski, Alexander Kaplan, Harshit Khaitan, Daniel Killebrew, Andy Koch, Naveen Kumar, Steve Lacy, James Laudon, James Law, Diemthu Le, Chris Leary, Zhuyuan Liu, Kyle Lucke, Alan Lundin, Gordon MacKean, Adriana Maggiore, Maire Mahony, Kieran Miller, Rahul Nagarajan, Ravi Narayanaswami, Ray Ni, Kathy Nix, Thomas Norrie, Mark Omer-nick, Narayana Penukonda, Andy Phelps, Jonathan Ross, Matt Ross, Amir Salek, Emad Samadiani, Chris Severn, Gregory Sizikov, Matthew Snelham, Jed Souter, Dan Steinberg, Andy Swing, Mercedes Tan, Gregory Thorson, Bo Tian, Horia Toma, Erick Tuttle, Vijay Vasudevan, Richard Walter, Walter Wang, Eric Wilcox, and Doe Hyun Yoon. 2017. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*. 1–12.
- [22] Andrew B. Kahng, Bill Lin, and Siddhartha Nath. 2015. ORION3.0: A comprehensive NoC router estimation tool. *IEEE Embedded Systems Letters* 7, 2 (2015), 41–45.
- [23] Myeonggu Kang, Hyein Shin, and Lee-Sup Kim. 2022. A framework for accelerating transformer-based language model on ReRAM-Based architecture. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41, 9 (2022), 3026–3039.
- [24] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. 2020. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 7871–7880.
- [25] Huize Li, Hai Jin, Long Zheng, Yu Huang, and Xiaofei Liao. 2022. ReCSA: A dedicated sort accelerator using ReRAM-based content addressable memory. *Frontiers of Computer Science* 17, 2 (2022), 172103.
- [26] Huize Li, Hai Jin, Long Zheng, Xiaofei Liao, Yu Huang, Cong Liu, Jiahong Xu, Zhuohui Duan, Dan Chen, and Chuangyi Gui. 2023. CPSAA: Accelerating sparse attention using crossbar-based processing-in-memory architecture. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2023), 1–1.
- [27] Weitao Li, Pengfei Xu, Yang Zhao, Haitong Li, Yuan Xie, and Yingyan Lin. 2020. Timely: Pushing data movements and interfaces in PIM accelerators towards local and in time domain. In *Proceedings of the 2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. 832–845.
- [28] Cong Liu, Haikun Liu, Hai Jin, Xiaofei Liao, Yu Zhang, Zhuohui Duan, Jiahong Xu, and Huize Li. 2022. ReGNN: A ReRAM-based heterogeneous architecture for general graph neural networks. In *Proceedings of the 59th ACM/IEEE Design Automation Conference (DAC)*. 469–474.

- [29] Haikun Liu, Jiahong Xu, Xiaofei Liao, Hai Jin, Yu Zhang, and Fubing Mao. 2022. A simulation framework for memristor-based heterogeneous computing architectures. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41, 12 (2022), 5476–5488.
- [30] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A robustly optimized BERT pretraining approach. *CoRR* abs/1907.11692 (2019). arXiv:1907.11692
- [31] Haiyu Mao, Mingcong Song, Tao Li, Yuting Dai, and Jiwu Shu. 2018. LerGAN: A zero-free, low data movement and PIM-based GAN architecture. In *Proceedings of the 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 669–681.
- [32] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An imperative style, high-performance deep learning library. In *Proceedings of the Advances in Neural Information Processing Systems (NIPS)*. 8024–8035.
- [33] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2383–2392.
- [34] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Fei-Fei Li. 2015. ImageNet large scale visual recognition challenge. *International Journal of Computer Vision* 115, 3 (Apr. 2015), 211–252.
- [35] Ali Shafiee, Anirban Nag, Naveen Muralimanohar, Rajeev Balasubramanian, John Paul Strachan, Miao Hu, R. Stanley Williams, and Vivek Srikumar. 2016. ISAAC: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars. In *Proceedings of the 43rd International Symposium on Computer Architecture (ISCA)*. 14–26.
- [36] Karen Simonyan and Andrew Zisserman. 2014. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556* (2014).
- [37] Linghao Song, Xuehai Qian, Hai Li, and Yiran Chen. 2017. PipeLayer: A pipelined ReRAM-based accelerator for deep learning. In *Proceedings of the 2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 541–552.
- [38] Linghao Song, Youwei Zhuo, Xuehai Qian, Hai Li, and Yiran Chen. 2018. GraphR: Accelerating graph processing using ReRAM. In *Proceedings of 2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 531–543.
- [39] Shrihari Sridharan, Jacob R. Stevens, Kaushik Roy, and Anand Raghunathan. 2023. X-Former: In-memory acceleration of transformers. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 31, 8 (2023), 1223–1233.
- [40] Sheng-Yang Sun, Hui Xu, Jiwei Li, Qingjiang Li, and Haijun Liu. 2019. Cascaded architecture for memristor crossbar array based larger-scale neuromorphic computing. *IEEE Access* 7 (2019), 61679–61688.
- [41] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. 2021. Training data-efficient image transformers & distillation through attention. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, Vol. 139. 10347–10357.
- [42] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Proceedings of the Advances in Neural Information Processing Systems (NIPS)*, Vol. 30. 1–11.
- [43] Yitu Wang, Zhenhua Zhu, Fan Chen, Mingyuan Ma, Guohao Dai, Yu Wang, Hai Li, and Yiran Chen. 2021. ReREC: In-ReRAM acceleration with access-aware mapping for personalized recommendation. In *Proceedings of the 2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. 1–9.
- [44] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander Rush. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, 38–45.
- [45] Wei Wu, Huaqiang Wu, Bin Gao, Peng Yao, Xiang Zhang, Xiaochen Peng, Shimeng Yu, and He Qian. 2018. A methodology to improve linearity of analog RRAM for neuromorphic computing. In *Proceedings of the 2018 IEEE Symposium on VLSI Technology*. 103–104.
- [46] Jiahong Xu, Haikun Liu, Zhuohui Duan, Xiaofei Liao, Hai Jin, Xiaokang Yang, Huize Li, Cong Liu, Fubing Mao, and Yu Zhang. 2024. ReHarvest: An ADC resource-harvesting crossbar architecture for ReRAM-based DNN accelerators. *ACM Transactions on Architecture and Code Optimization* 21, 3, Article 63 (Sept. 2024), 26 pages.
- [47] Bonan Yan, Yuchao Yang, and Ru Huang. 2023. Memristive dynamics enabled neuromorphic computing systems. *Science China Information Sciences* 66, 10 (2023), 200401.

- [48] Chao Yang, Xiaoping Wang, and Zhigang Zeng. 2022. Full-circuit implementation of transformer network based on memristor. *IEEE Transactions on Circuits and Systems I: Regular Papers* 69, 4 (2022), 1395–1407.
- [49] Tzu-Hsien Yang, Hsiang-Yun Cheng, Chia-Lin Yang, I-Ching Tseng, Han-Wen Hu, Hung-Sheng Chang, and Hsiang-Pang Li. 2019. Sparse ReRAM Engine: Joint exploration of activation and weight sparsity in compressed neural networks. In *Proceedings of the International Symposium on Computer Architecture (ISCA)*. 236–249.
- [50] Xiaoxuan Yang, Bonan Yan, Hai Li, and Yiran Chen. 2020. ReTransformer: ReRAM-based processing-in-memory architecture for transformer acceleration. In *Proceedings of the 2020 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. 1–9.
- [51] Amir Yazdanbakhsh, Ashkan Moradifirouzabadi, Zheng Li, and Mingu Kang. 2022. Sparse attention acceleration with synergistic in-memory pruning and on-chip recomputation. In *Proceedings of 2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 744–762.
- [52] Abdullah Serdar Yonar, Pier Andrea Francese, Matthias Brändli, Marcel Kossel, Mridula Prathapan, Thomas Morf, Andrea Ruffino, and Taekwang Jang. 2023. An 8b 1.0-to-1.25GS/s 0.7-to-0.8V single-stage time-based gated-ring-oscillator ADC with 2× interpolating sense-amplifier-latches. In *Proceedings of the 2023 IEEE International Solid-State Circuits Conference (ISSCC)*. 1–3.
- [53] Geng Yuan, Payman Behnam, Zhengang Li, Ali Shafiee, Sheng Lin, Xiaolong Ma, Hang Liu, Xuehai Qian, Mahdi Nazm Bojnordi, Yanzhi Wang, and Caiwen Ding. 2021. FORMS: Fine-grained polarized ReRAM-based in-situ computation for mixed-signal DNN accelerator. In *Proceedings of the 2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. 265–278.

Received 14 June 2024; revised 2 September 2024; accepted 29 September 2024